

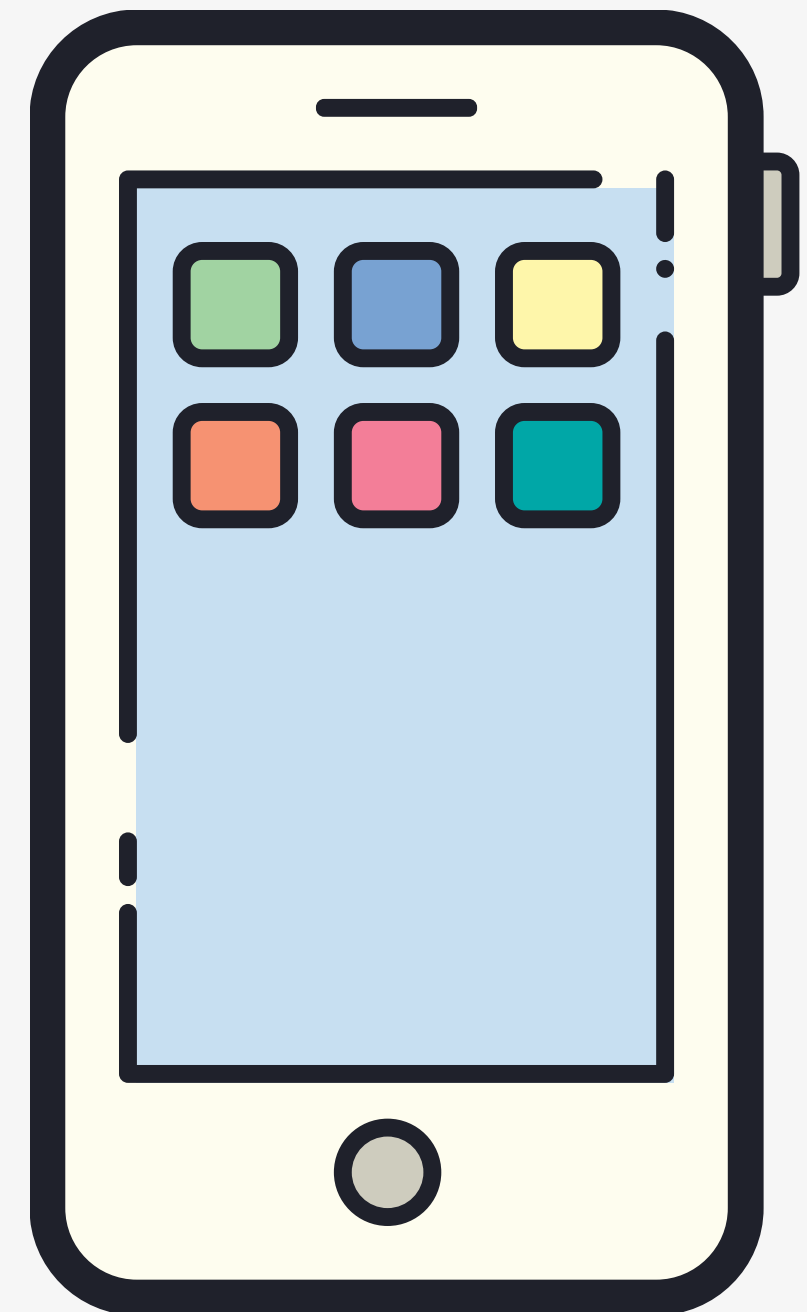
SmartContacts: Personal Contact Manager

Group 4: Daniel Nguyen, Anthony Pham, Wei Yuan, Sam Bess, Connor Linscott, Alejandro Appio-Riley, Danial Parviz

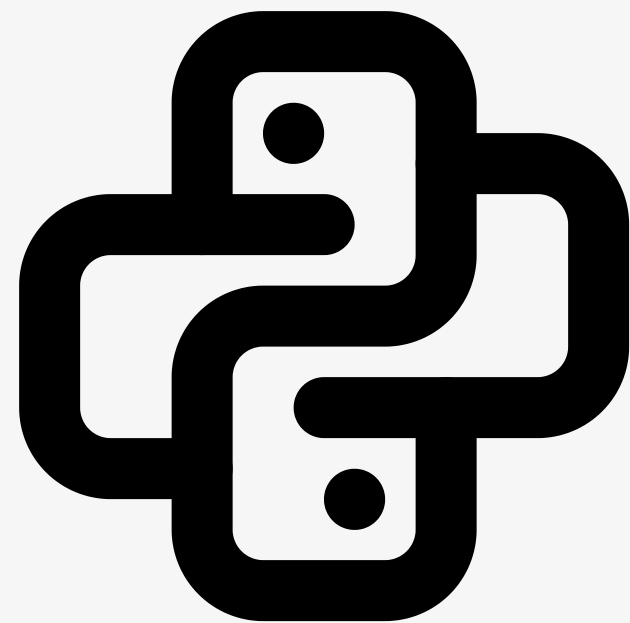


Objective

- Provide users with a centralized system to manage contact information
 - Store and edit
 - Organize and search
 - Data portability
- Practical alternative to basic phone contact apps offering organization features and better data control through a web browser with an intuitive interface



Proposed Tech Stack



Python/Flask



SQLite

`{.js}`
JavaScript

JavaScript/HTML/CSS

Features

- Add, remove, edit (CRUD)
- Advanced search, filtering, and grouping
- Import/Export for CSV files
- Blacklist contacts



Cost Estimation

Function Point Analysis

Scope Requirement	FP Type	Count	Weight	FP Total
3.1.1 Add Contact	EI	1	3	3
3.1.1 Edit Contact	EI	1	3	3
3.1.1 Delete Contact	EI	1	3	3
3.1.1 View Contact	EQ	1	3	3
3.1.3 Add Photo Label (upload)	EI	1	3	3
3.1.3 Display Photo	EO	1	4	4
3.1.4 Search Contacts (query + display results)	EQ	1	3	3
3.1.5 Add to Blacklist	EI	1	3	3
3.1.5 Remove from Blacklist	EI	1	3	3
3.1.5 Blacklist Table	ILF	1	7	7
3.1.6 Make Phone Call / Send SMS	EO	1	4	4
3.2.1 Add Contact Group	EI	1	3	3
3.2.2 Add Contact to Group	EI	1	3	3
3.2.2 Remove Contact from Group	EI	1	3	3
3.2.3 Sort Contacts (name/group)	EO	1	4	4
3.3.1 Export Contacts	EO	1	4	4
3.3.2 Load Contacts from File (import)	EI	1	3	3
Contacts Data File	ILF	1	10	10
Groups Data File	ILF	1	7	7

$FP = EI + EO + EQ + ILF + EIF$

$FP = 30 + 16 + 6 + 24 + 0 = 76$ Function Points

Critical Path Method

Note: Some of the critical path method is on the project timeline slide (next slide)

Product size: 76 FP → 760 person-hours (10 hours per FP).

Workday: 8 hours.

Team & billing rates: Backend Dev \$70/hr, Frontend Dev \$70/hr, QA \$50/hr, DevOps \$80/hr, PM \$90/hr, Business Analyst (BA) \$65/hr.

Activity	Hours	Primary role(s)	Labor cost
Requirements	38	BA + PM	$(30 \times 65) + (8 \times 90) = 1,950 + 720 = \$2,670$
Design	76	Backend + Frontend + PM	$(20 \times 70) + (30 \times 70) + (26 \times 90) = 1,400 + 2,100 + 2,340 = \$5,840$
Database Schema	61	Backend	$61 \times 70 = \$4,270$
Backend API	228	Backend	$228 \times 70 = \$15,960$
Frontend UI	190	Frontend	$190 \times 70 = \$13,300$
Integration	53	Backend+ Frontend	$(30 \times 70) + (23 \times 70) = \$3,710$
Testing	76	QA (+devs)	$76 \times 50 + 10 \times 70 + 10 \times 70 = 3,800 + 700 + 700 = \$5,200$
Deployment	23	DevOps	$23 \times 80 = \$1,840$
Training & Docs	15	PM + Backend	$(8 \times 90) + (7 \times 70) = 720 + 490 = \$1,210$

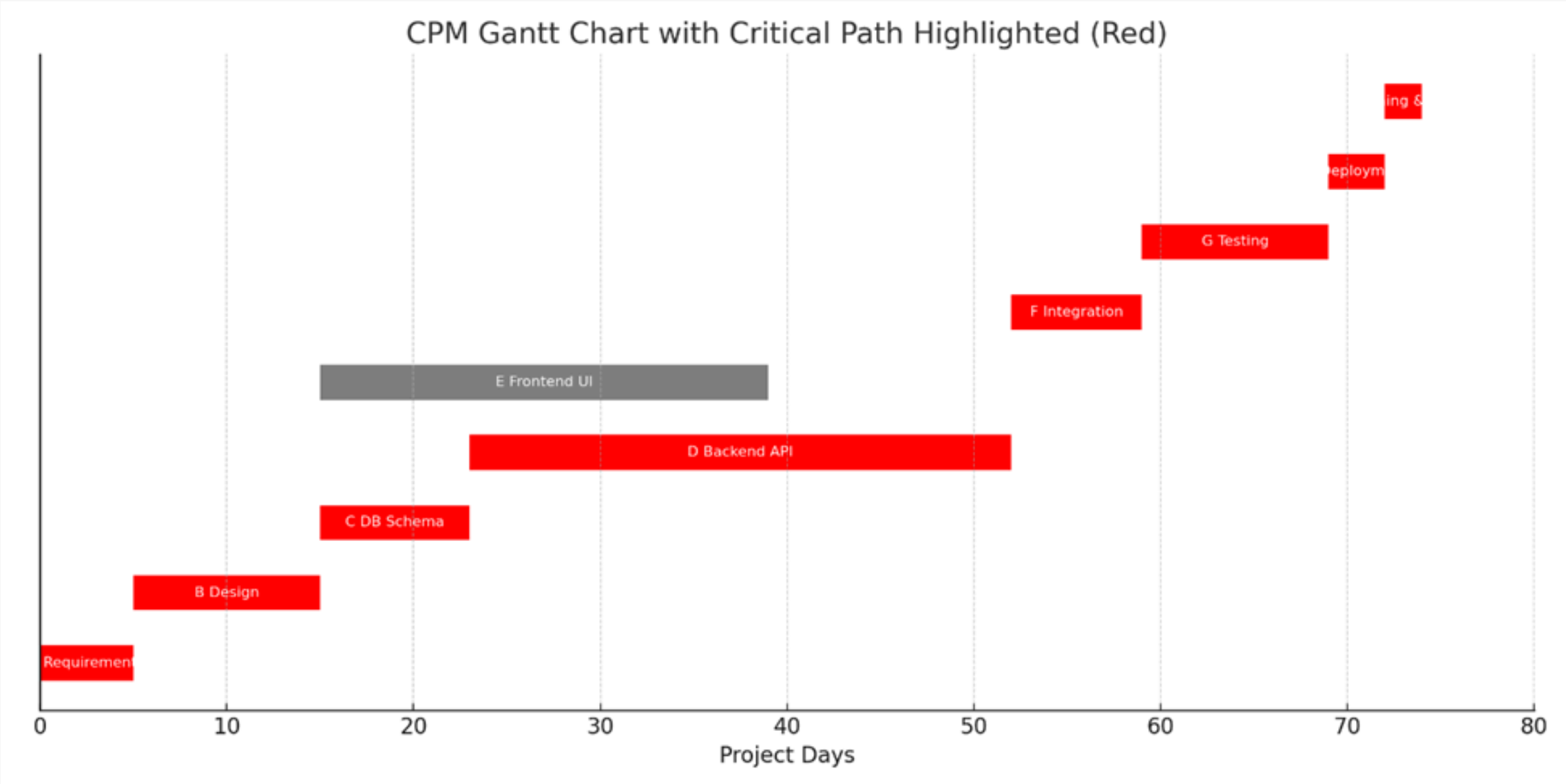
Labor subtotal: 760 h \$54,000

Hardware subtotal: Cloud hosting & infrastructure (annual estimate, e.g. small AWS stack): \$1,284

Software subtotal: Software / Licenses / SSL / Monitoring (annual): \$1,275

Total Direct costs: \$56,559

Project Timeline



Critical path (zero float tasks): A → B → C → D → F → G → H → I.

Frontend UI (E) is not on the critical path and has 13 days float (it can slip up to 13 working days without delaying the project).

Project duration: 74 working days (Mon–Fri) → ~15 weeks.

Critical Path Method: Activities, effort, durations, and dependencies

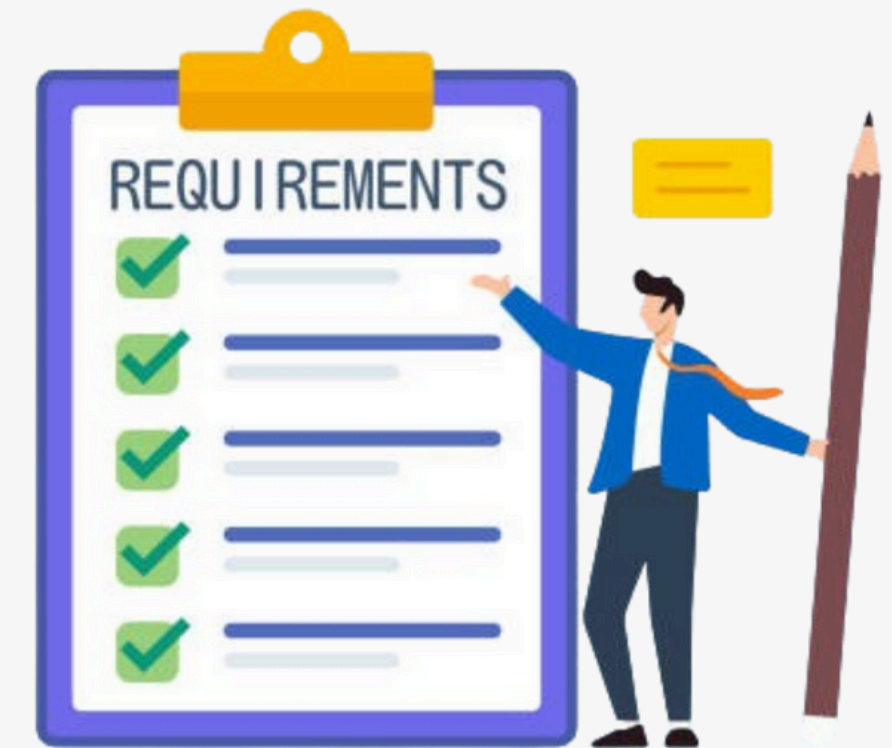
ID	Activity	Hours	Duration (working days)	Predecessors
A	Requirements / Analysis	38	5	—
B	Design (UI + System)	76	10	A
C	Database Schema & Model	61	8	B
D	Backend API & Services	228	29	C
E	Frontend UI Implementation	190	24	B
F	Integration (Backend + Frontend)	53	7	D, E
G	Testing / QA	76	10	F
H	Deployment / DevOps	23	3	G
I	Training & Documentation	15	2	H

Critical Path Method: Early/Late times, float, and the critical path

Activity	ES	EF	LS	LF	Float	Duration (days)
A Requirements	0	5	0	5	0	5
B Design	5	15	5	15	0	10
C Database Schema	15	23	15	23	0	8
D Backend API	23	52	23	52	0	29
E Frontend UI	15	39	28	52	13	24
F Integration	52	59	52	59	0	7
G Testing	59	69	59	69	0	10
H Deployment	69	72	69	72	0	3
I Training & Docs	72	74	72	74	0	2

Functional Requirements

- Add a new contact with required fields.
- Edit and update existing contact details.
- Delete a contact from their list after confirmation.
- Search and filter contacts by name or category.
- Display a clear list of all contacts stored in the database.
- Validate user input and prevent saving incomplete contact entries.



Non-Functional Requirements

Product Requirements

- System loads contact list within 2 seconds
- Supports 100+ contacts without lag
- Stores only essential data fields
- Automatically saves data after each action
- Protects contact data from unauthorized access



Non-Functional Requirements Cont.

Organizational

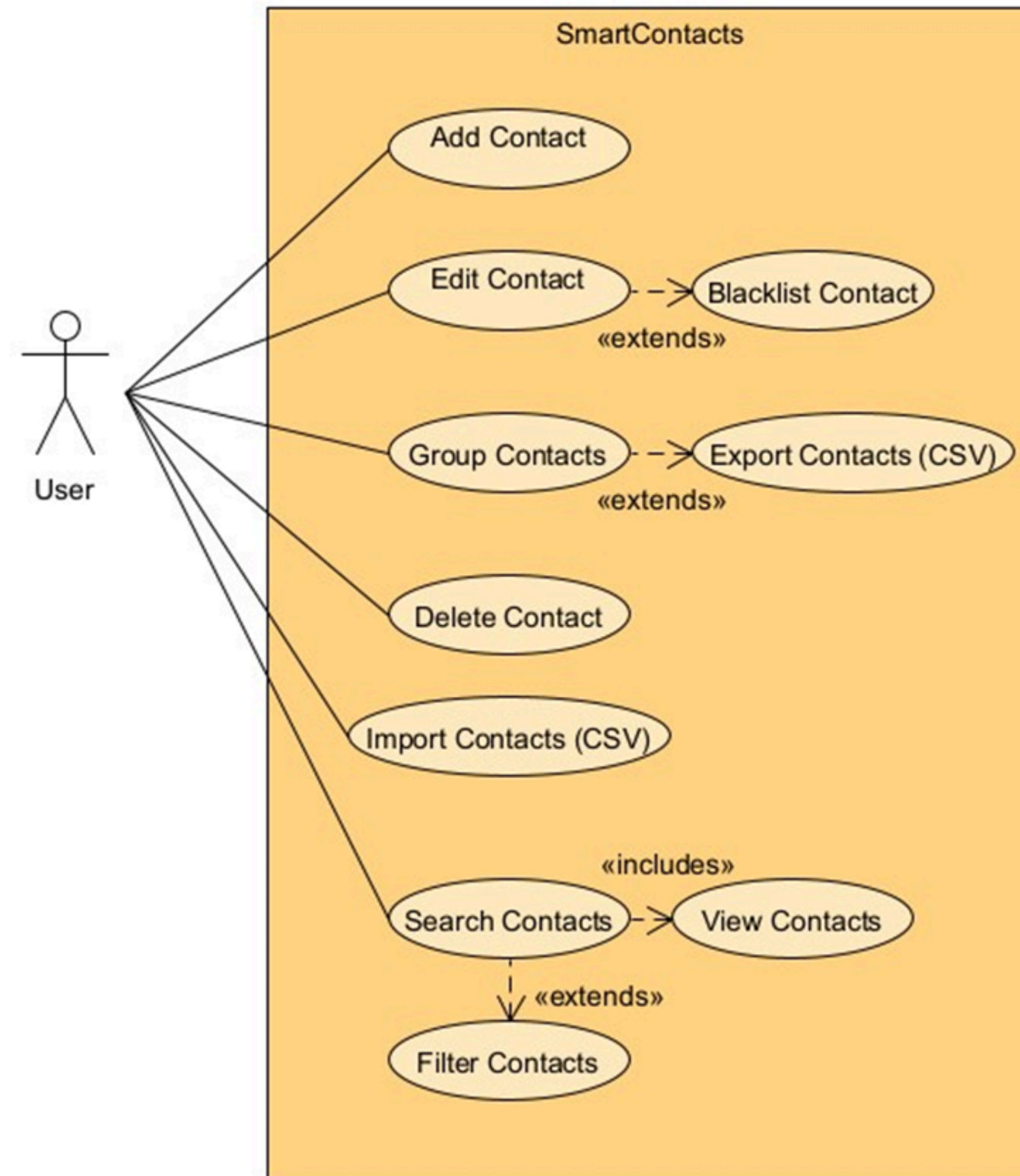
- Must run on modern web browsers
- Follows MVC architecture
- Code must be clean and well-commented

External

- Privacy respected
- No financial or payment data stored
- Confirmation required before deleting data

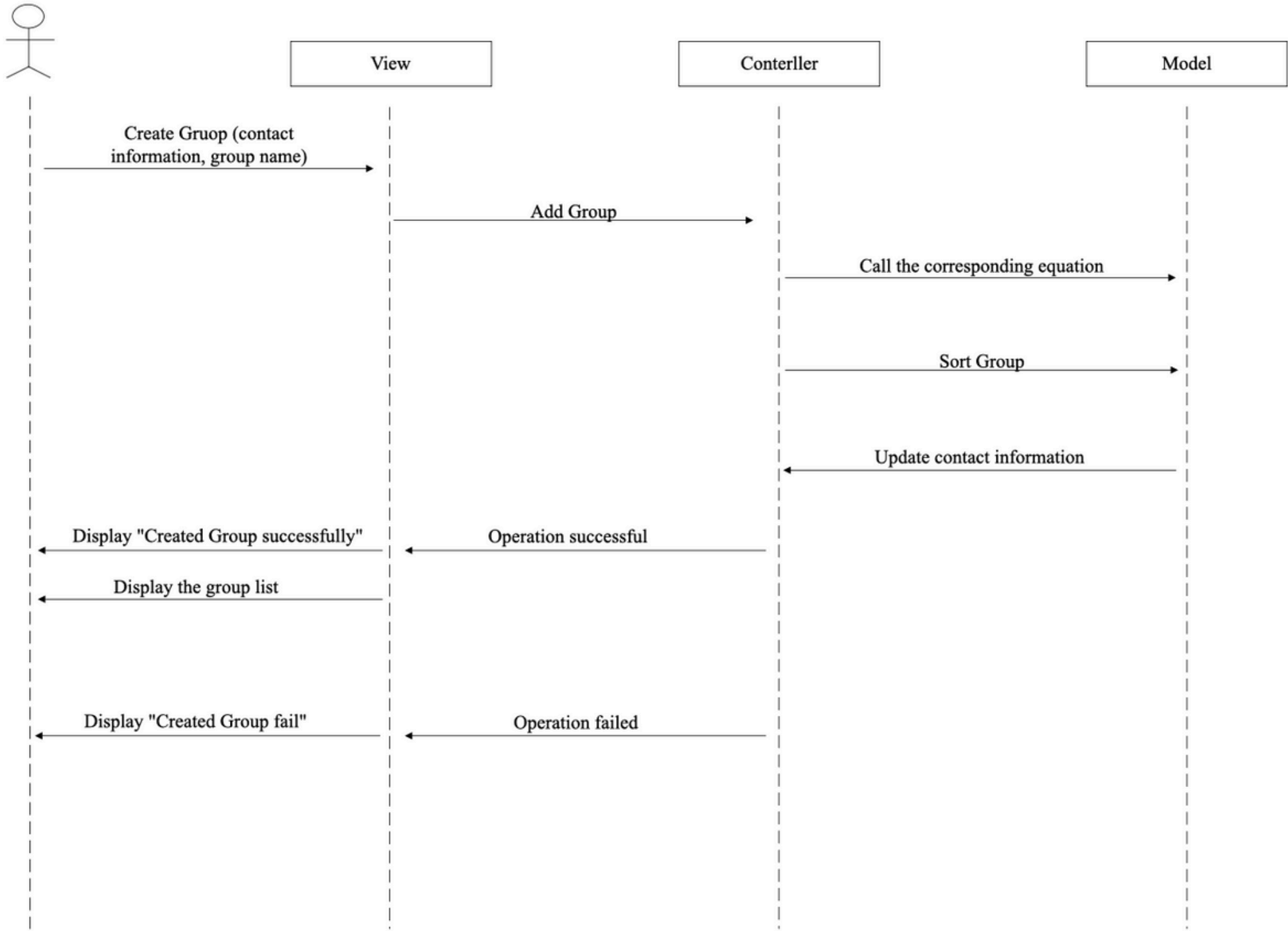


Use Case Diagram

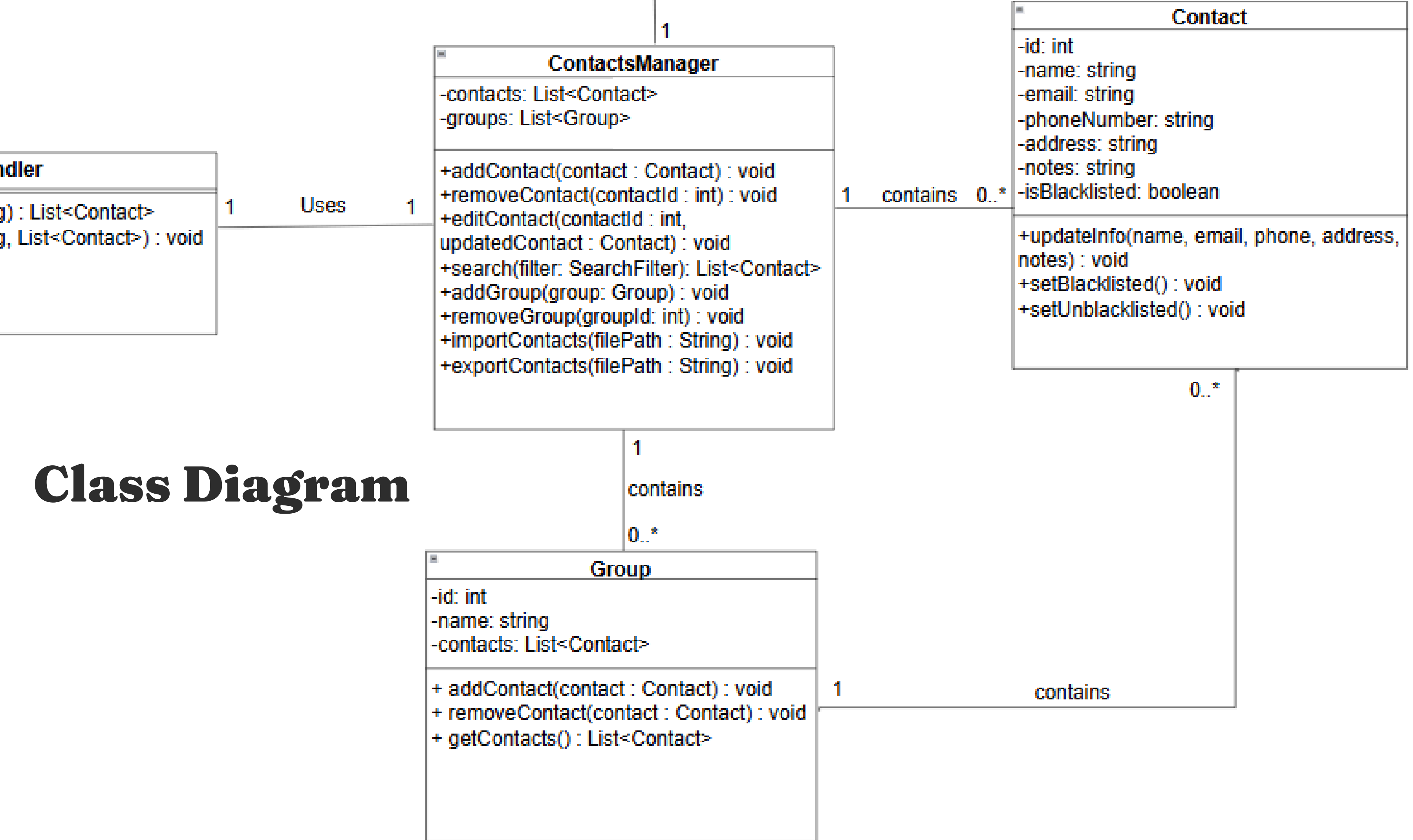


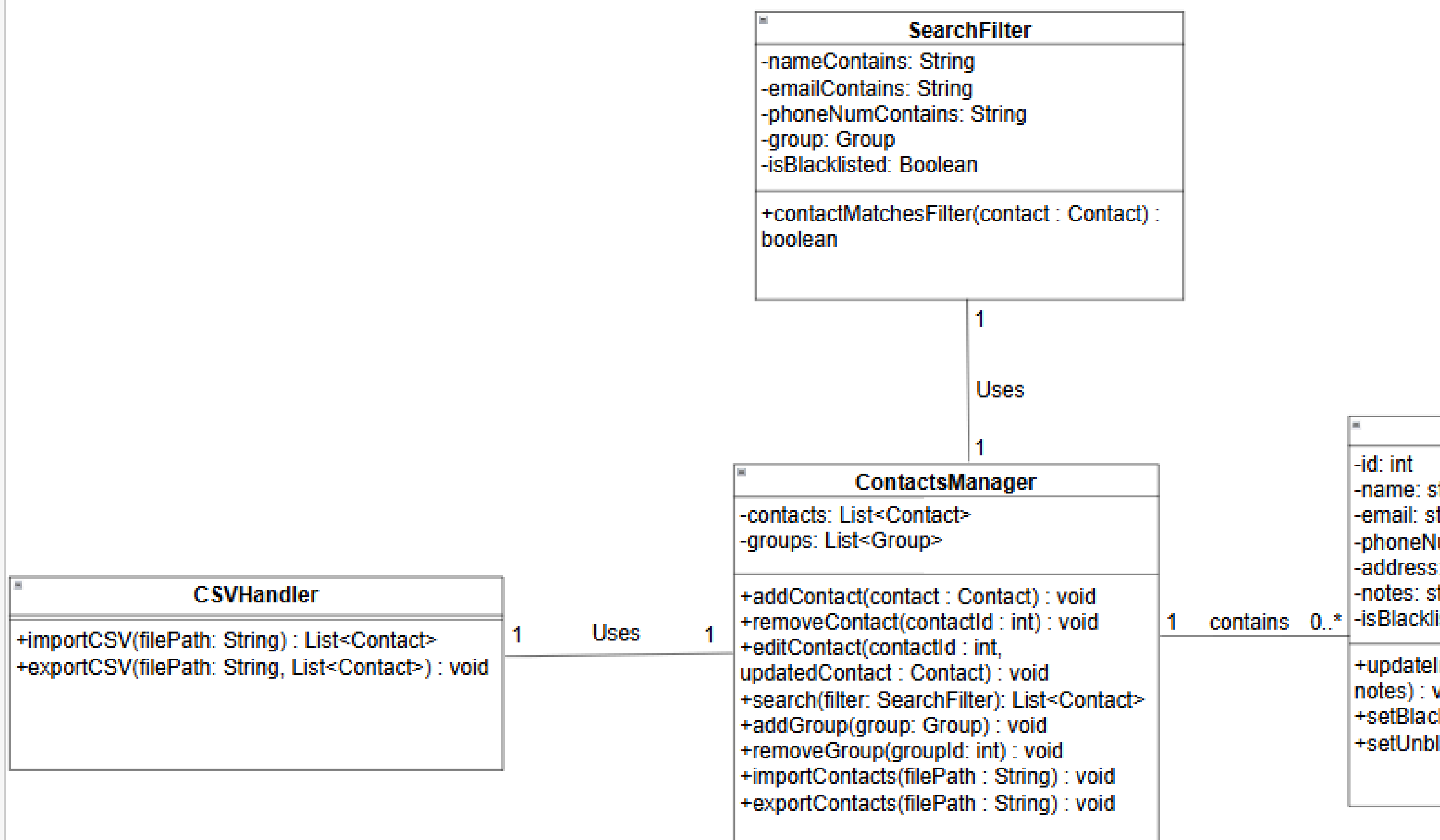
Sequence Diagram

Sequence diagram for Grouping contact



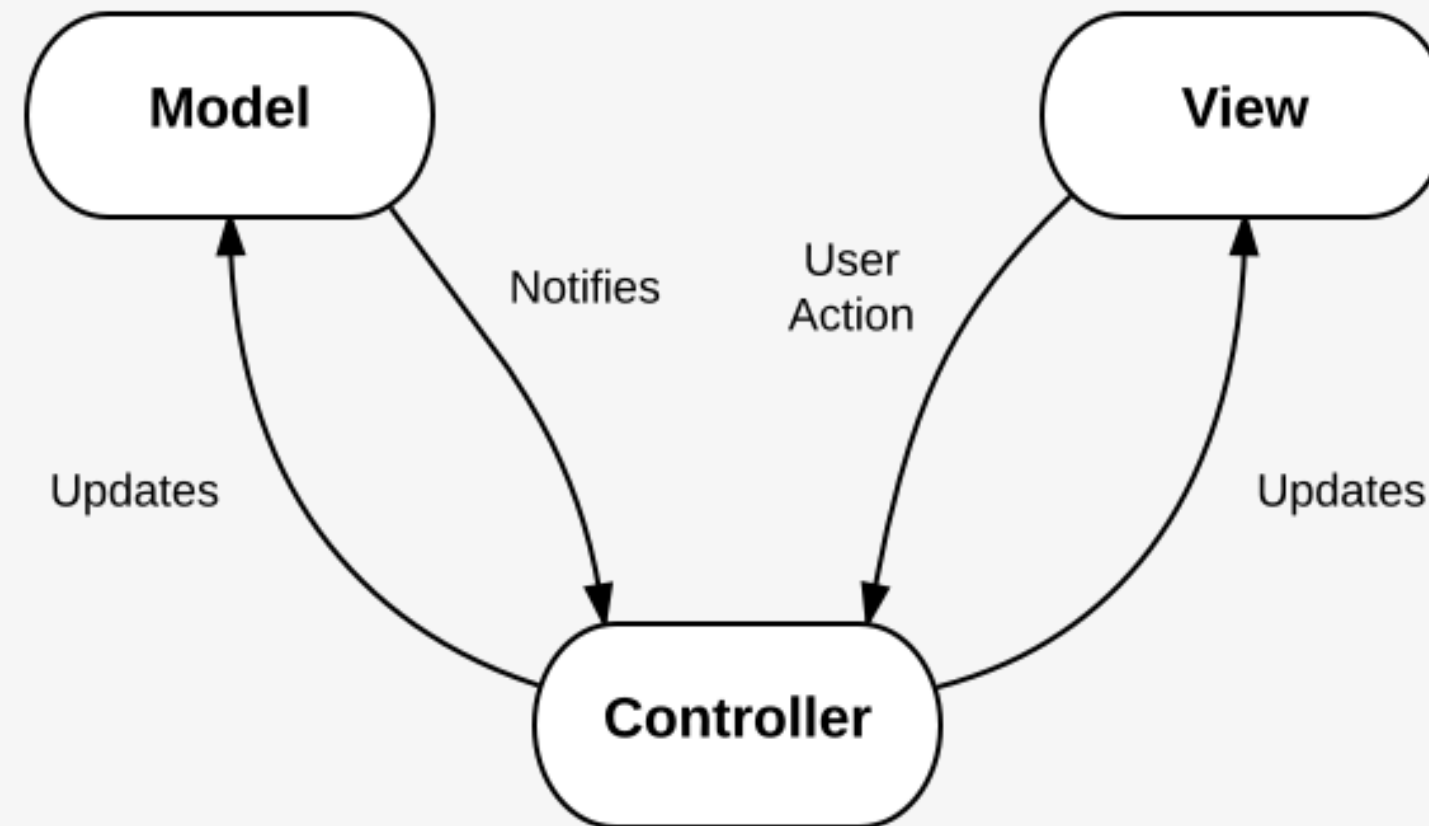
Class Diagram





Architectural Design

- Handles all data + database actions
- Stores and updates contact information



- Displays contacts and forms
- Presents results to the user

- Handles user actions / logic (add, edit, delete)
- Sends updates to Model + refreshes View

Test Plane ----- add account unit

Check

Check whether the add account unit works successfully and if working in correct way as expected

Help

Help identify whether there are any errors that need to be fixed

Ensure

Ensure the quality of the project

Simple code

```
AccountTest.java Account.java Main.java X
1 package org.junit;
2
3 import java.util.*;
4 import java.util.Scanner;
5
6
7 public class Main
8 {
9     public static void main(String[] args) {
10         Scanner input = new Scanner(System.in);
11         ArrayList<Account> account = new ArrayList<>();
12
13         while(true)
14         {
15             System.out.print("Do you want to add new account? (yes / no): ");
16             String choice = input.nextLine();
17
18             if(choice.equals("yes") || choice.equals("Yes"))
19             {
20                 System.out.print("Name: ");
21                 String name = input.nextLine();
22
23                 System.out.print("Phone Number: ");
24                 String phoneNumber = input.nextLine();
25
26                 System.out.print("Group Type: ");
27                 String groupType = input.nextLine();
28
29                 account.add(new Account(name, phoneNumber, groupType));
30             }
31
32             else if (choice.equals("no") || choice.equals("No"))
33             {
34                 break;
35             }
36
37             else
38             {
39                 System.out.print("Please enter yes or no: ");
40             }
41         }
42
43         System.out.println("***** The List of contant account *****");
44         System.out.printf("%-25s %-20s %-15s\n", "Account Name", "Phone Number", "Group Type");
45
46         for(int i = 0; i < account.size(); i++)
47         {
48             Account n = account.get(i);
49             System.out.printf("%-25s %-20s %-15s\n", n.getName(), n.getPhoneNumber(), n.getGroupType());
50         }
51     }
52 }
53 }
```

Console X

<terminated> Main [Java Application] /Users/a.../.p2/pool/plugins/org.eclipse.justj.open

Do you want to add new account? (yes / no): yes

Name: alince

Phone Number: 1231231234

Group Type: aa

Do you want to add new account? (yes / no): yes

Name: ben

Phone Number: 1231231234

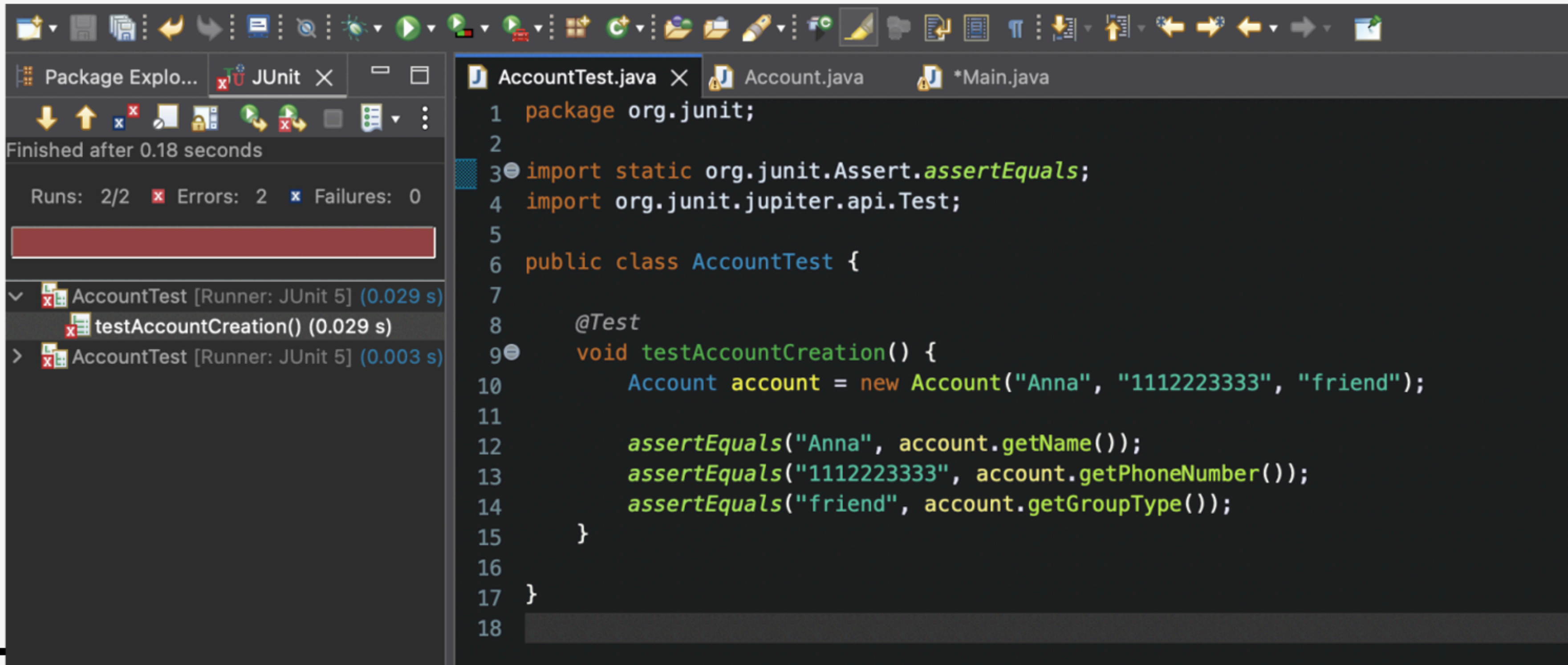
Group Type: dd

Do you want to add new account? (yes / no): no

***** The List of contant account *****

Account Name	Phone Number	Group Type
alince	1231231234	aa
ben	1231231234	dd

Testing code and result



The screenshot displays an IDE interface with a code editor and a test results panel. The code editor shows the following Java code:

```
1 package org.junit;  
2  
3 import static org.junit.Assert.assertEquals;  
4 import org.junit.jupiter.api.Test;  
5  
6 public class AccountTest {  
7  
8     @Test  
9     void testAccountCreation() {  
10         Account account = new Account("Anna", "1112223333", "friend");  
11  
12         assertEquals("Anna", account.getName());  
13         assertEquals("1112223333", account.getPhoneNumber());  
14         assertEquals("friend", account.getGroupType());  
15     }  
16  
17 }  
18
```

The test results panel on the left shows the following information:

- Package Explo... JUnit
- Finished after 0.18 seconds
- Runs: 2/2
- Errors: 2
- Failures: 0
- AccountTest [Runner: JUnit 5] (0.029 s)
- testAccountCreation() (0.029 s)
- AccountTest [Runner: JUnit 5] (0.003 s)

Test cases

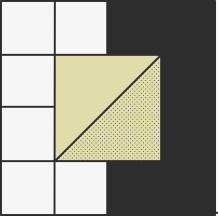
Test Case	Input	Actual Value	Expected Value	Result
Name	Anna	account.getName()	Anna	✗
Phone number	1112223333	account.getPhoneNumber()	1112223333	✗
Group type	friend	account.getGroupType()	friend	✗

Possible reasons

The wrong code of testing or wrong way

The environment differences

- The input path is different.



Thank you!

